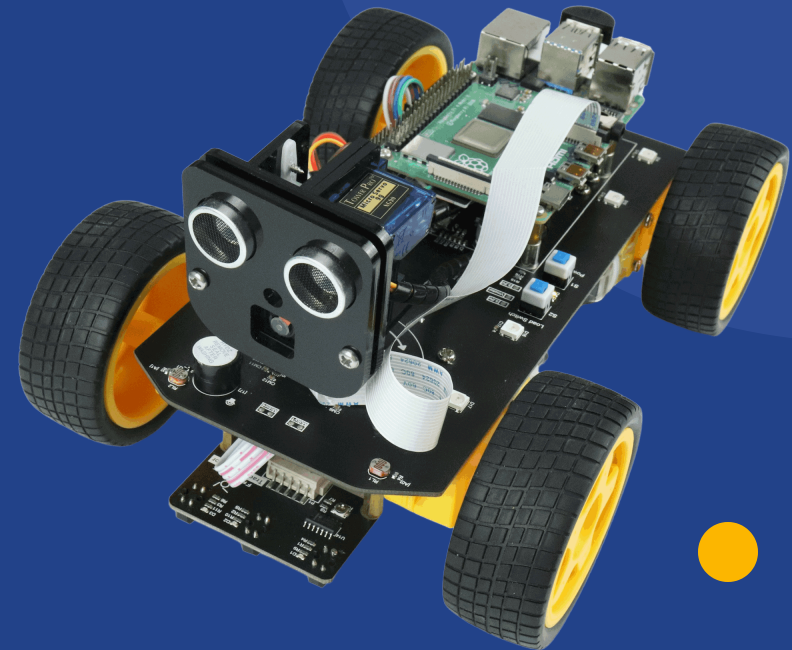




LESSON 6 – CAMERA STREAMING

Machine Learning with Raspberry Pi & Smart Car

The car already drives and senses. Today we give it eyes – live video from the Pi's camera, streamed to your computer over Wi-Fi.





LESSON 6 – CAMERA STREAMING

Now we give the car eyes

So far the car drives and senses. Today it starts to **see** – and we send what it sees to your screen.

- So far the car **drives** and **senses** – now we give it **eyes**.
- We'll send live video from the Pi's camera to your computer over **Wi-Fi**.
- This is the foundation for **face tracking** in Lessons 7 & 8.



Camera → over Wi-Fi → your screen

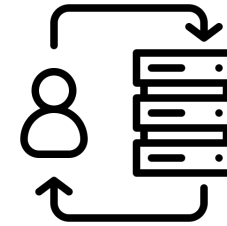


LESSON 6 – CAMERA STREAMING

Today's goal: two programs, one video

Build **two programs** that talk to each other across a network.

- 1 **Server** (on the Pi) – opens the camera and sends pictures.
- 2 **Viewer / client** (on the Pi or a laptop) – shows the live video.
- 3 Learn how programs send data over a **network** with sockets.



Two files this lesson – keep clear **which one runs where.**



LESSON 6 – CAMERA STREAMING

Client & server, again

Same idea as Lesson 2 – but now over a real network instead of inside one program.



The **Pi is the server** – it waits on `port 8000` and sends video.



Your **computer is the client** – it connects and displays the video.

WHAT'S A PORT?

A **port** is like a numbered door on a computer.

The Pi opens door `8000` and waits. Your computer knocks on it to start the stream.



LESSON 6 – CAMERA STREAMING • THE KEY IDEA

The frame protocol: size, then picture

A connection only carries bytes. So both sides agree on one rule for **every frame**.

1 Send **4 bytes** = how big the picture is.

2 Send the **JPEG picture** bytes.



The same rule repeats for every frame, until a **size of 0** ends the stream.

A size of **0** means "**no more pictures – stop.**"



This rule is the contract between the **two files** we'll build: the **server** `lesson_6_pi_camera_stream_server.py` (on the Pi) sends frames, and the **client** / **viewer** `lesson_6_client_video_receiver.py` (Pi or laptop) reads them.



SERVER • CHUNK 1 / 7

Server: imports

`picamera2` controls the Pi camera. The rest are Python's standard tools.

- `io` in-memory file • `socket` network.
- `struct` turns a number into bytes • `time`.
- `picamera2` controls the Raspberry Pi camera.

```
lesson_6_pi_camera_stream_server.py

"""Pi side: stream the camera to the computer."""

import io          # an in-memory "file" for the photo
import socket      # talk over the network
import struct      # number -> fixed bytes
import time

from picamera2 import Picamera2 # the Pi camera
```



SERVER • CHUNK 2 / 7

Server: settings

`VIDEO_PORT = 8000` is the door the computer connects to.

- **Smaller frames** stream more smoothly – try this later.
- Picture size, frame rate, and a short camera **warm-up**.
- `HOST = ""` means accept a connection on any address.

```
lesson_6_pi_camera_stream_server.py

# ---- settings ----
VIDEO_PORT = 8000      # door the computer connects to
HOST = ""              # accept any of our addresses
FRAME_WIDTH = 400      # picture width in pixels
FRAME_HEIGHT = 300     # picture height in pixels
FRAME_RATE = 15        # pictures per second
WARMUP_SECONDS = 2     # let the camera adjust
```



SERVER • CHUNK 3 / 7

setup(): open the door, start the camera

Make a socket, claim port 8000, **listen** – then start the camera and let it warm up.

- Make a **door**, claim a **number**, **listen**.
- `SO_REUSEADDR` lets us restart cleanly.
- Then `camera.start()` and a short warm-up.

```
lesson_6_pi_camera_stream_server.py

server_socket = None # waits for the computer
connection = None   # the pipe we write pictures into
camera = None

def setup():
    """Open the network doorway and start the camera."""
    global server_socket, camera
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((HOST, VIDEO_PORT)) # claim the port
    server_socket.listen(1)               # wait for 1 connection

    camera = Picamera2()
    camera.configure(camera.create_video_configuration(
        main={"size": (FRAME_WIDTH, FRAME_HEIGHT)}))
    camera.start()
    time.sleep(WARMUP_SECONDS)             # let it settle
```



SERVER • CHUNK 4 / 7

accept_client(): wait for the computer

`accept()` pauses the program until the computer connects.

- `accept()` **pauses** until the computer connects.
- `makefile("wb")` lets us **write bytes** like a file.
- `"wb"` = write-bytes; the viewer will use `"rb"`.

```
lesson_6_pi_camera_stream_server.py

def accept_client():
    """Wait for the computer, then open the pipe."""
    global connection
    client_socket, address = server_socket.accept()
    connection = client_socket.makefile("wb")
    print("Computer connected from", address[0])
```



SERVER • CHUNK 5 / 7

loop(): capture & send each frame

Take a JPEG into memory, then send **size first, picture second** – the protocol from slide 5.

- Capture one JPEG into an in-memory file.
- `struct.pack("<L", length)` makes the 4 size-bytes.
- Then write the picture bytes after the size.

```
lesson_6_pi_camera_stream_server.py

def loop():
    """Capture forever; send each picture down the pipe."""
    stream = io.BytesIO()          # in-memory file
    while True:
        stream.seek(0); stream.truncate() # empty it
        camera.capture_file(stream, format="jpeg") # one JPEG

        length = stream.tell()        # how many bytes
        connection.write(struct.pack("<L", length)) # size
        connection.flush()             # push it out now

        stream.seek(0)                # back to the start
        connection.write(stream.read()) # the picture bytes
```



SERVER • CHUNK 6 / 7

destroy(): send "done" & close

Send a size of `0` as the agreed stop signal, then close everything tidily.

- A size of `0` tells the viewer to stop.
- The computer may already be gone – wrap it in `try`.
- Close the camera and the socket cleanly.

```
lesson_6_pi_camera_stream_server.py

def destroy():
    """Tell the computer we're done and close up."""
    if connection is not None:
        try:
            connection.write(struct.pack("<L", 0)) # 0 = stop
        except OSError:
            pass # computer may be gone
        connection.close()
    if camera is not None:
        camera.stop(); camera.close()
    if server_socket is not None:
        server_socket.close()
```



SERVER • CHUNK 7 / 7

Server: the main block

Open everything, wait for the computer, then stream – with the usual safe wrapper.

- Order: `setup()` → `accept_client()` → `loop()`.
- The server **waits** for a viewer before sending anything.
- `finally` always runs `destroy()` – same as every lesson.

```
lesson_6_pi_camera_stream_server.py

if __name__ == "__main__":
    try:
        setup()
        accept_client()      # wait for the computer
        loop()               # then stream pictures
    except KeyboardInterrupt:
        pass
    finally:
        destroy()
```



VIEWER • CHUNK 1 / 5

Viewer: imports

First time using `cv2` (OpenCV) – your tool for everything visual.

- `socket`, `struct`, `sys` (read the IP you type).
- `cv2` (OpenCV) shows the picture in a window.
- `numpy` turns raw bytes into an image.

```
lesson_6_client_video_receiver.py

"""Viewer side: show the Pi's camera video."""

import socket # connect to the Pi
import struct # size-bytes -> a number
import sys    # read the IP you type

import cv2     # OpenCV: shows the window
import numpy as np # bytes -> an image
```



VIEWER • CHUNK 2 / 5

setup(): read the IP & connect

You type the Pi's IP on the command line; `sys.argv[1]` reads it. Port must match the server.

- Connect to port `8000` – same as the server.
- `makefile("rb")` = read-bytes like a file.
- Use `127.0.0.1` if the viewer runs on the Pi.

```
lesson_6_client_video_receiver.py

VIDEO_PORT = 8000          # match the Pi server
WINDOW_NAME = "Lesson 6 - Pi Camera"
video_socket = None        # our connection to the Pi
video_file = None          # the same connection, read like a file

def setup():
    """Read the Pi's address and connect."""
    global video_socket, video_file
    if len(sys.argv) < 2:
        raise SystemExit("Pass the Pi's IP, e.g. python this.py 192.168.1.50")
    pi_ip = sys.argv[1]

    video_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    video_socket.connect((pi_ip, VIDEO_PORT))    # dial the Pi
    video_file = video_socket.makefile("rb")     # read bytes
```



VIEWER • CHUNK 3 / 5

read_frame(): size, then picture

This mirrors the server exactly: server **packs** the size, viewer **unpacks** it.

- Read the **4 size-bytes**, turn them into a number.
- If the size is `0` → the stream ended.
- Else read that many bytes and decode the JPEG.

```
lesson_6_client_video_receiver.py

def read_frame():
    """Read one picture, or None when the stream ends."""
    header = video_file.read(4)          # the 4 size-bytes
    if len(header) != 4:
        return None
    length = struct.unpack("<L", header)[0] # bytes -> number
    if length == 0:
        return None                      # the Pi's "done" signal

    jpeg = video_file.read(length)        # exactly that many bytes
    if len(jpeg) != length:
        return None
    return cv2.imdecode(np.frombuffer(jpeg, np.uint8), cv2.IMREAD_COLOR)
```



VIEWER • CHUNK 4 / 5

loop(): show the video

Read a frame, show it, repeat. Press `q` to quit.

- `read_frame()` returns `None` → the stream ended.
- `imshow` draws the picture in the window.
- `waitKey(1)` refreshes it and checks for `q`.

```
lesson_6_client_video_receiver.py

def loop():
    """Show pictures until the video ends or you quit."""
    while True:
        frame = read_frame()
        if frame is None:
            break # no more pictures
        cv2.imshow(WINDOW_NAME, frame) # draw it
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
```



VIEWER • CHUNK 5 / 5

destroy() + the main block

Close the connection and the window cleanly – the same safe pattern as always.

- Close the file and the socket.
- `destroyAllWindows()` shuts the OpenCV window.
- Just closing a connection and a window – not motors.

```
lesson_6_client_video_receiver.py

def destroy():
    """Close the connection and the window."""
    if video_file is not None:
        video_file.close()
    if video_socket is not None:
        video_socket.close()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    try:
        setup(); loop()
    except KeyboardInterrupt:
        pass
    finally:
        destroy()
```

LESSON 6 – CAMERA STREAMING

You built both files

Two programs, one rule they both agree on: **size first, picture second**.



Server (Pi)

```
imports → settings → setup() → accept_client() →  
loop() → destroy()
```



Viewer (computer)

```
imports → setup() → read_frame() → loop() →  
destroy()
```



The symmetry: the server `pack` s the size, the viewer `unpack` s it – server `"wb"` , viewer `"rb"` .

LESSON 6 – CAMERA STREAMING

Run it

Server first, then viewer. The viewer needs the Pi's IP address.

Step 1 – on the Pi, start the server

 Pi terminal

```
cd Smart-Car-Level-3/Code/User  
sudo python3 lesson_6_pi_camera_stream_server.py
```

Step 2 – on the Pi or your laptop, start the viewer

 terminal

```
sudo python lesson_6_client_video_receiver.py 192.168.1.50
```



Find the Pi's IP with `hostname -I`. Use `127.0.0.1` on the Pi.



A window shows the live camera. Press `q` to quit.



LESSON 6 – CAMERA STREAMING

Try it yourself

Experiment with the settings – smaller frames stream much more smoothly over Wi-Fi.

- Change `FRAME_WIDTH` / `FRAME_HEIGHT` (server) – bigger vs smoother?
- Change `FRAME_RATE` and watch the smoothness change.
- Rename `WINDOW_NAME` (viewer) to your own title.

CHALLENGE

Two computers, one camera

Run the viewer on a **laptop** while a classmate's runs on the **Pi** – both watching the same stream.



LESSON 6 – CAMERA STREAMING

Common issues

If the stream looks off, it's almost always one of these.



Connection refused

Start the **server first**, and check the IP / port (`8000`).



Black or frozen window

Camera not ready – check the camera cable and `picamera2` .



Laggy video

Lower the resolution / frame rate, or get closer to the router.



ModuleNotFoundError: cv2

Install `opencv-python` and `numpy` on the viewer.



Check this first

90% of issues are **viewer started before server** or a **wrong IP**. Check those two first.



LESSON 6 – CAMERA STREAMING

The car can stream what it sees!

You moved the course from *driving* to *seeing* – with a socket and a simple frame protocol.

- ✓ Streamed live video from the Pi using a **socket**
- ✓ Agreed on a simple **frame protocol** – size, then picture
- ✓ Met **OpenCV** – your tool for everything visual



UP NEXT

Lesson 7 Face tracking

Detect a **face** in the video and turn the head to follow it – reusing this exact `read_frame()` idea.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Confirm the cameras work

Check each **camera module** is connected and `picamera2` works on every Pi.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance